boundary for this first step.

Let us start with the first source area you recommended: geocoding.
For now, do not try to force a pattern name onto it. I want to understand the collaboration first.
Inspect these files and any directly related callers you need:

1. `app/models/concerns/geocodable.rb`
2. `app/models/trip_item.rb`
3. `app/jobs/geocode_trip_items_job.rb`
4. `app/controllers/trips_controller.rb`
5. `config/initializers/geocoder.rb`

Trace the geocoding flow from the point where the application triggers it up to the external geocoding boundary.
For this investigation:

1. Identify the actual classes, modules and methods involved.
2. Show how control moves between them.
3. Separate code written by DaPlanStan from behavior supplied by Rails and the Geocoder dependency.
4. Record the exact source paths and reliable line ranges for the important interactions.
5. Explain the design problem this collaboration is solving.
6. Only after tracing the collaboration, list any design pattern that could reasonably describe it.
7. For each possible pattern, explain both why it might fit and why it might be a false positive.
8. Do not confirm anything yet. Mark each possible classification as a candidate only.
9. If the collaboration is simply shared behavior, framework usage or ordinary delegation rather than a recognized pattern, say so.

At the end give me:

1. The complete geocoding collaboration trace.
2. The relevant source evidence.
3. Candidate pattern classifications, if any.
4. The strongest alternative explanation.

5. What still needs to be checked before any candidate can be confirmed.

Worked for 1m 40s

Geocoding Collaboration — Source Investigation 01

Candidates only DaPlanStan original source ZIP · Static source analysis

1. Main finding

The geocoding implementation has **three distinct triggering paths**, rather than a single controller-to-job workflow.

1. **Manual request:** A user selects *Fix missing pins* on the trip page. `TripsController` identifies records without coordinates and schedules `GeocodeTripItemsJob`.
2. **Bulk import:** `TripItemsController` imports itinerary items, temporarily disables the model's geocoding callback, then schedules the same background job for newly created records needing coordinates.
3. **Ordinary record saving:** The `Geocodable` concern registers an Active Record `after_save` callback. This can invoke geocoding during a normal `TripItem` save without passing through `GeocodeTripItemsJob`.

These paths ultimately reach the same application-level operation: `TripItem#geocode_from_address`, which delegates to the implementation supplied by the `Geocodable` module. ‹/› ‹/›

This distinction matters because **not every geocoding operation is asynchronous**. The manual and bulk-import paths enqueue a job, whereas the model callback is part of the normal record-save lifecycle.

The strongest initial explanation is shared model behavior combined with Rails callbacks, background-job orchestration and a Geocoder library integration. A recognized GoF pattern has not yet been demonstrated.

2. Complete geocoding collaboration trace

Control-flow overview